



ReFAIR

Toward a Context-Aware Recommender
for Fairness Requirements Engineering

Pre-Maintenance Report

Università degli studi di Salerno

Corso di Laurea Magistrale in Informatica

Corso

di

Ingegneria, Gestione ed Evoluzione del Software

Riferimento	Report
Versione	1.0
Data	04/02/2025
Presentato da	PASTORE Carmine LAURINO Nicola

Sommario

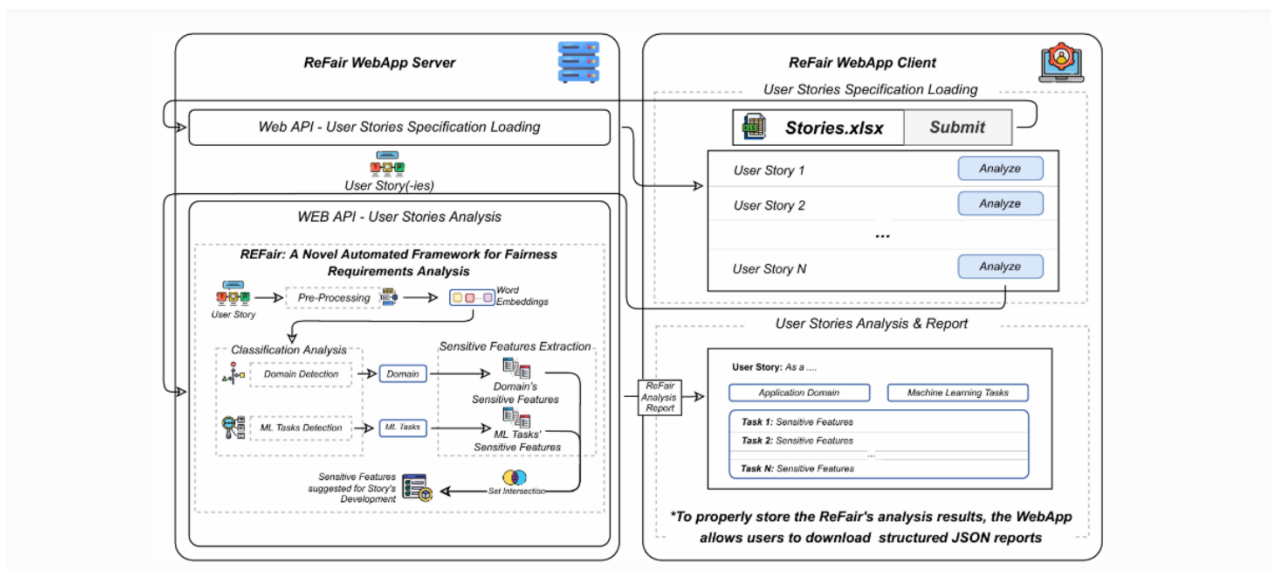
- 1. Introduzione 3
- 2. Architettura attuale 3
- 3. Reverse engineering 5
 - 3.1 Use case Analisi storia..... 5
 - 3.2 Sequence diagram 6
 - 3.3 Package diagram 7
 - 3.4 Activity diagram 8
 - 3.5 Class diagram..... 9
- 4. Criticità sistema attuale 10

1. Introduzione

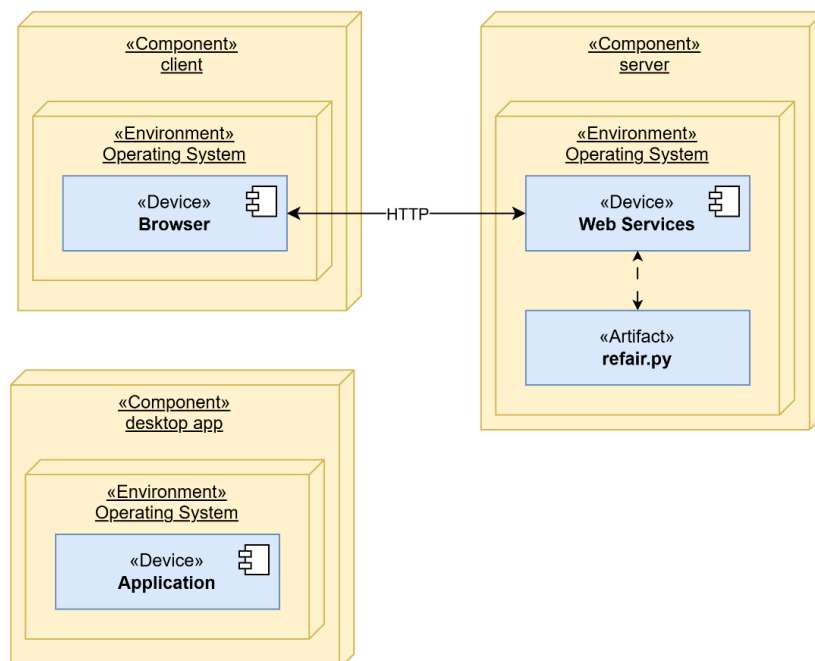
Il seguente documento ha il fine ultimo di descrivere lo stato attuale del sistema Refair prima dell'implementazione delle change requests sottomesse (salvo approvazione).

L'obiettivo principale del documento è fornire una panoramica sull'architettura esistente, i componenti principali del sistema e come avvengono i principali task del sistema, per comprendere al meglio il funzionamento del sistema sarà condotta una fase di reverse engineering, per capire al meglio il funzionamento dell'applicazione.

2. Architettura attuale



L'architettura attuale è organizzata secondo un modello client-server, e per il lato server presenta una struttura monolitica, che comporta diversi problemi in termini di manutenibilità.



Il sistema si compone di tre principali elementi:

- Server: il server è scritto in Python, utilizzando il framework Flask. Il server mette a disposizione del client le funzioni attraverso degli endpoint, che permettono la predizione del dominio, dei possibili task e e l'estrazione di feature sensibili mediante l'analisi della user story passata in input al modello.
- Client WebApp: il client permette di caricare un file "xlsx" contenente le US, e attraverso un pulsante permette l'analisi della singola US.
- Desktop app: un'applicazione desktop standalone, che offre funzionalità simili ma indipendenti dal client web. Questa app permette agli utenti di eseguire analisi localmente senza dipendere dal server remoto. Tuttavia, non è integrata direttamente con il sistema principale, il che può creare ridondanza nei processi e gestione separata dei dati.

3. Reverse engineering

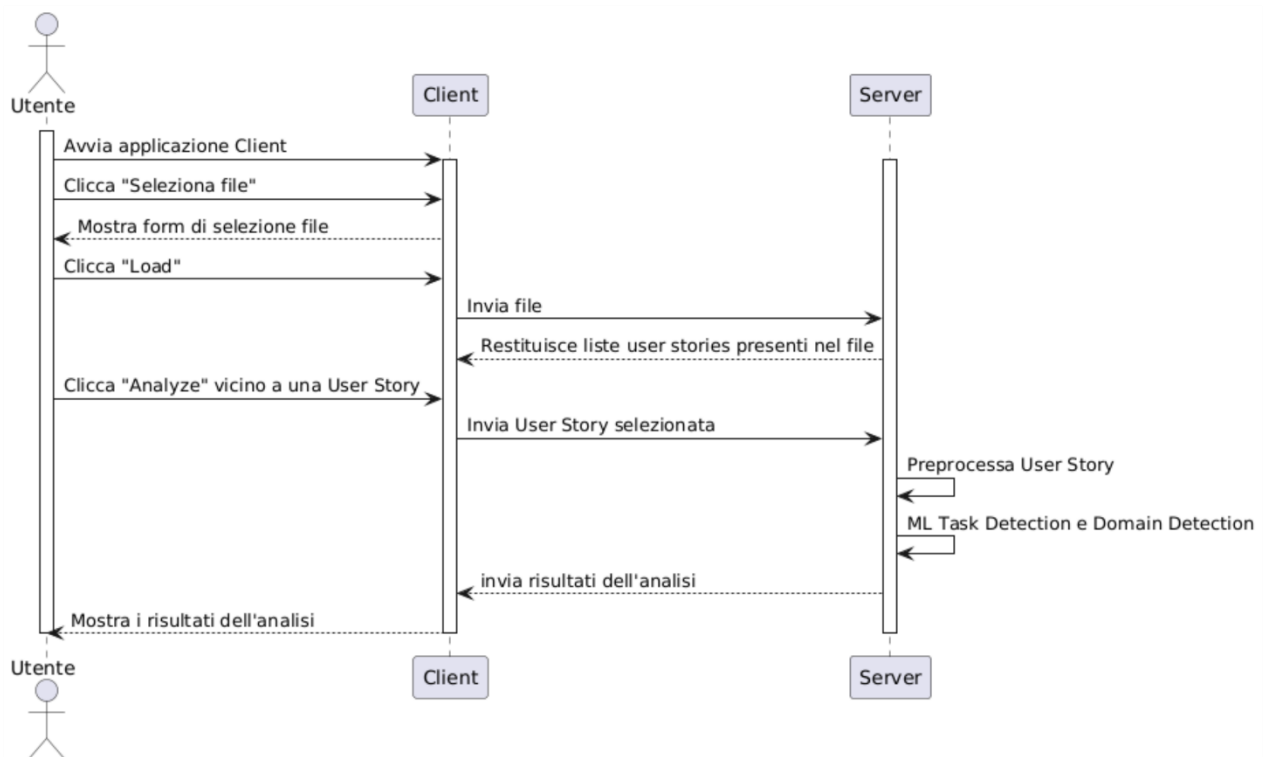
La fase di reverse engineering risulta fondamentale per comprendere nel dettaglio la logica e il funzionamento interno del sistema.

3.1 Use case Analisi storia

ID	UC_AN
Denominazione	Analisi stories.xlsx
Descrizione	Lo UC fornisce la funzionalità di effettuare l'analisi su una user story
Attore principale	Utente
Attori secondari	Webapp
Entry condition	L'utente ha avviato la web app.
Exit condition on success	L'utente visualizza l'analisi della user story
Exit condition on failure	L'utente non visualizza l'analisi della user story
Frequenza stimata	10 volte/gg
	Flusso di eventi principale
1	Utente: L'utente clicca sul pulsante "select file"
2	Sistema: Mostra la finestra per selezionare il file
3	Utente: Seleziona il file stories.xlsx
4	Utente: clicca ok
5	Utente clicca: Load
6	Sistema: il sistema mostra all'utente la lista di tutte le US presenti nel file.xlsx.
7	Utente: Clicca sul pulsante "analyze" vicino alla US che vuole analizzare.
8	Sistema: Il sistema analizza la US.
9	Sistema: Il sistema mostra una finestra con i risultati dell'analisi della US.
	Flusso di eventi alternativo: errore nella scelta del file
2.1	Utente: sceglie un file .json
2.2	Utente: clicca ok
2.3	Utente clicca: Load;
2.4	<i>Sistema: mostra un alert di errore</i>
	Flusso di eventi alternativo: errore nella predizione
8.1	Sistema: il sistema mostra un alert di errore.

3.2 Sequence diagram

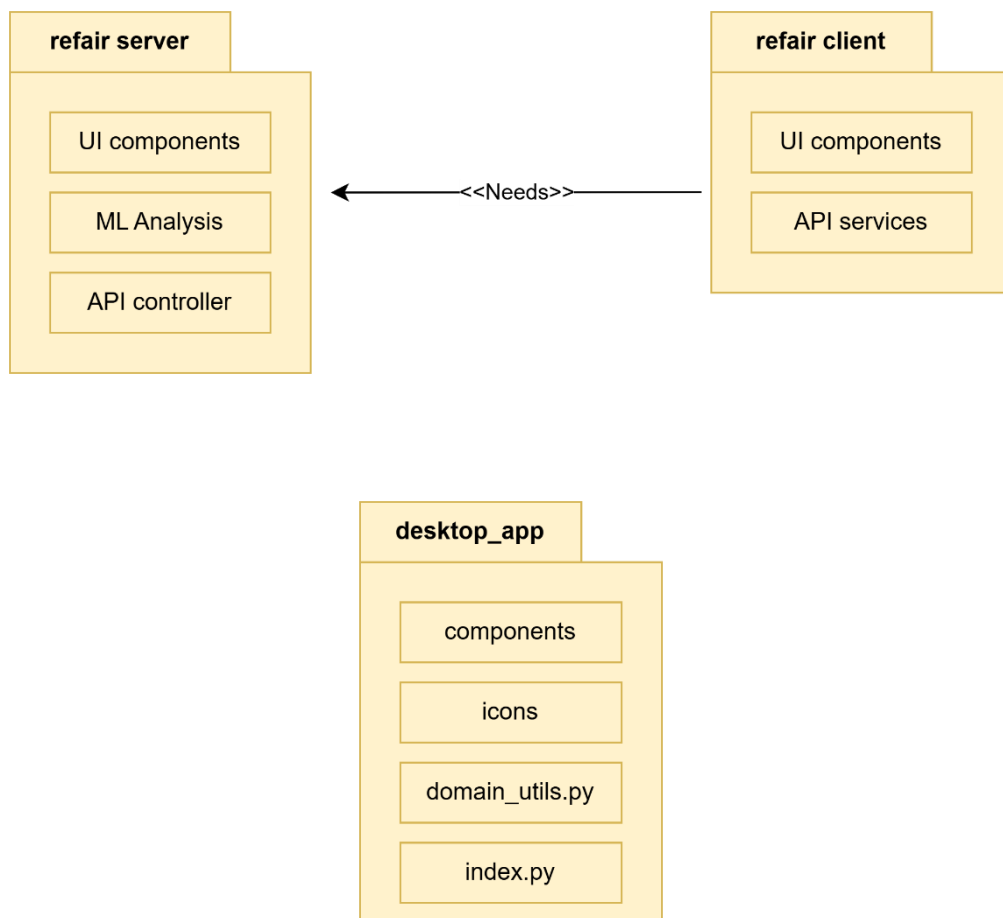
Nel seguente sequence diagram, andiamo a descrivere il flusso di interazioni che avviene tra utente, client e server, durante il processo di caricamento e analisi delle US, con lo scopo di mostrare le operazioni eseguite dall'utente e le comunicazioni tra il **Client WebApp** e il **Server Backend** per elaborare la richiesta di analisi. Il processo di interazione inizia con l'utente che avvia il client, e successivamente clicca sul pulsante per selezionare il file, lo seleziona e clicca sul pulsante "Load", in quel momento il file viene passato al server che ne estrapola tutte le US, presenti nel file, e le mostra all'utente. L'utente clicca sul pulsante "Analyze" vicino alla US che intende analizzare, e il client invia la richiesta al server, che esegue il **pre-processing** del testo e applica modelli di **Machine Learning** per rilevare il **dominio** e i **task ML** associati. Una volta completata l'analisi, il server restituisce i risultati al client, che li visualizza all'utente.



3.3 Package diagram

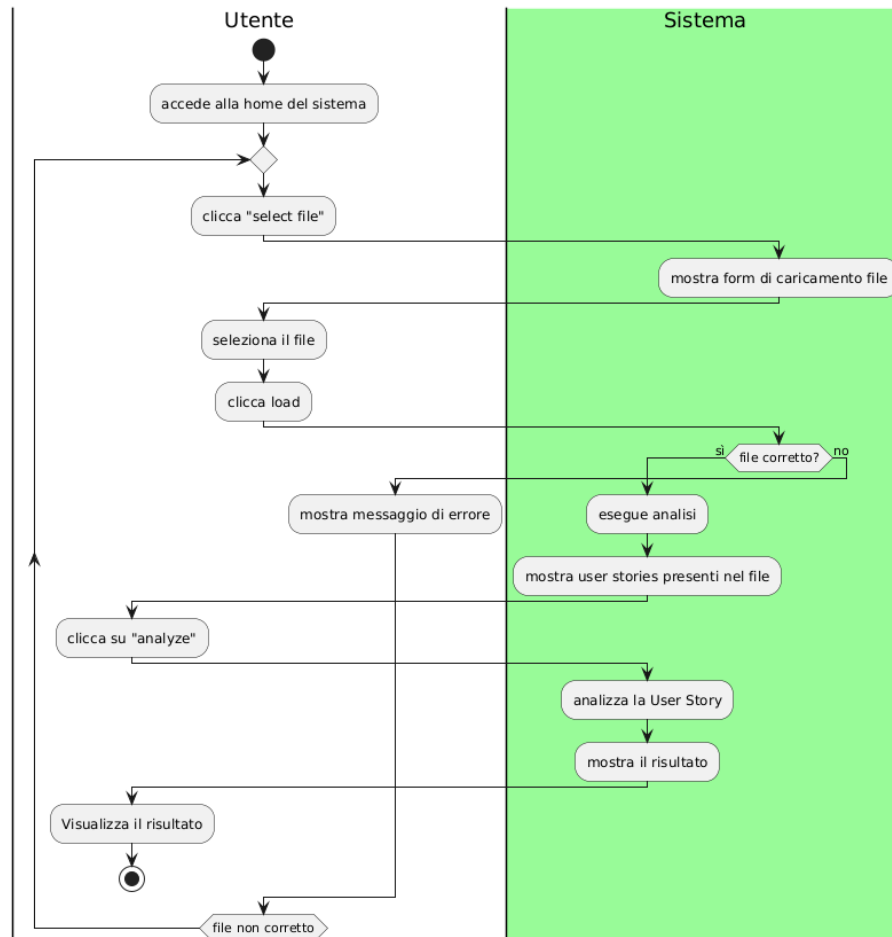
All'interno del sistema sono stati individuati tre package principali:

- Refair server: contiene gli endpoints per consentire alla webApp di interagire con il server e contiene i modelli di ML che si occupano di eseguire l'analisi delle US.
- Refair client: contiene interfaccia web e permette di inviare richieste al server.
- Desktop app: Un'applicazione autonoma che permette di eseguire analisi senza dipendere direttamente dal server.



3.4 Activity diagram

Il presente **Activity Diagram** rappresenta il flusso di attività che avviene durante il processo di **caricamento e analisi di una User Story** all'interno del sistema. Il diagramma mostra le interazioni tra l'**utente** e il **sistema**, evidenziando le decisioni e i percorsi alternativi in caso di errori.

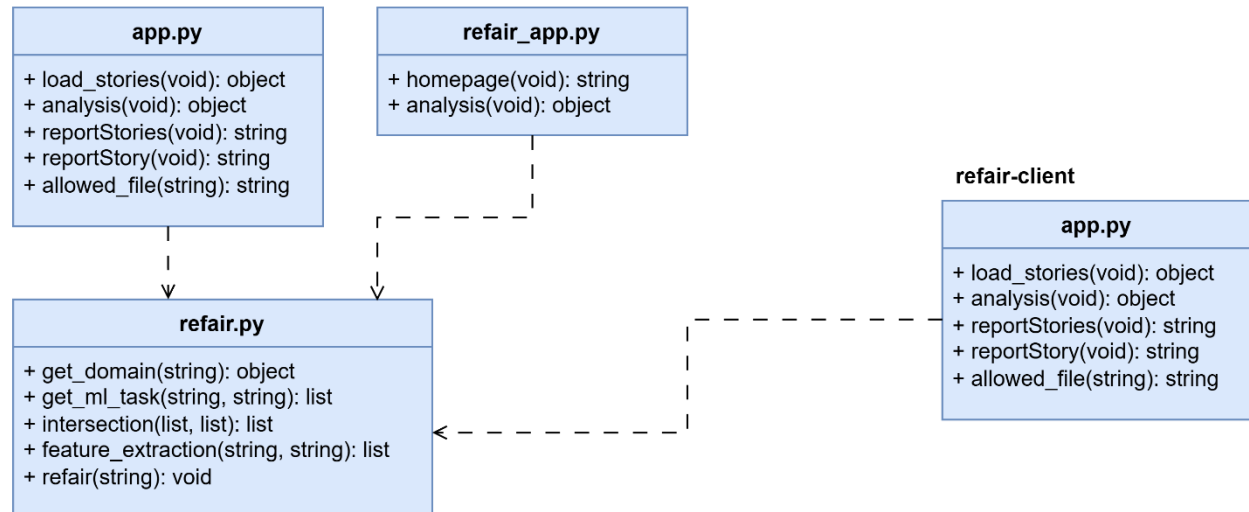


3.5 Class diagram

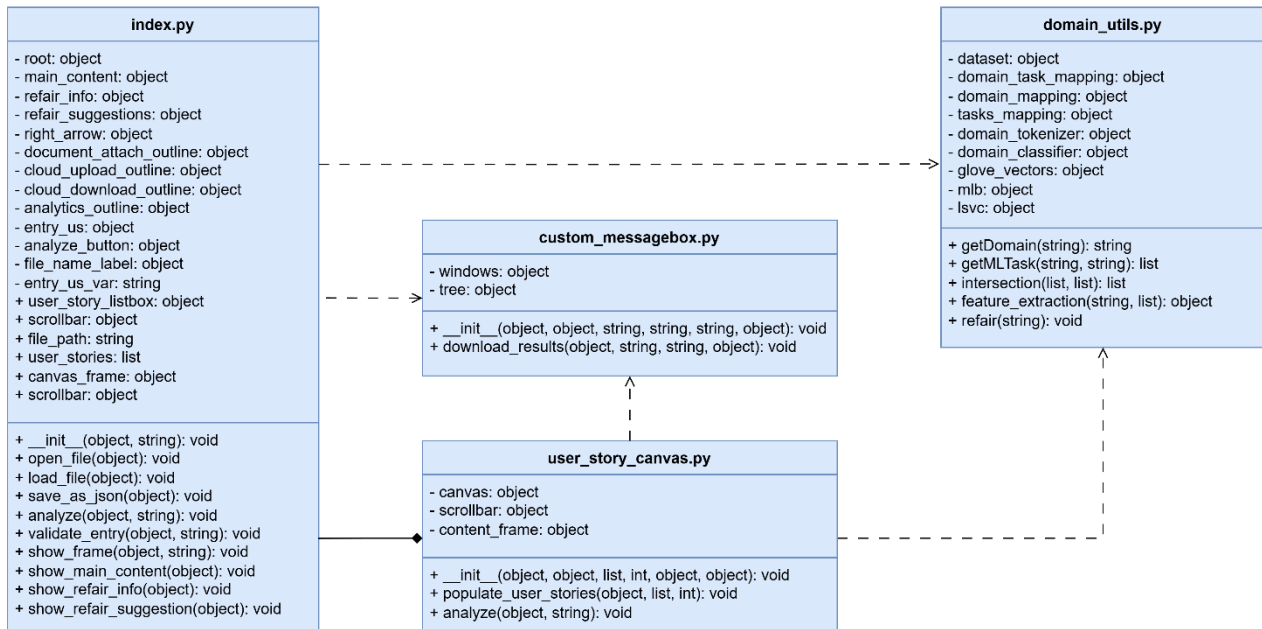
Il **Class Diagram** rappresenta la struttura del sistema, mostrando le classi principali, i loro attributi, metodi e relazioni. Questo schema aiuta a comprendere l'organizzazione del codice, evidenziando le dipendenze e le interazioni tra i vari moduli.

Possiamo notare che abbiamo due class diagram differenti, in quanto la desktop app è un'applicazione indipendente e non interagisce con il sistema client-server.

refair-server



desktop-app



4. Criticità sistema attuale

Dall'analisi condotta nella fase di **reverse engineering**, emergono diverse **criticità** che impattano la **scalabilità, la manutenibilità e le prestazioni** del sistema attuale. Questi problemi evidenziano la necessità di una ristrutturazione architetturale per migliorare l'efficienza complessiva del sistema. Di seguito ecco una lista delle principali criticità rilevate:

- Architettura Monolitica e Scarsa Modularizzazione
il backend è strutturato come un **sistema monolitico**, in cui tutti gli endpoint e le logiche di elaborazione sono accorpati in un'unica classe. Questo comporta difficoltà nella manutenzione, scalabilità limitata e scarsa separazione delle responsabilità rendendo il codice meno riutilizzabile.
- Ridondanza tra WebApp e Desktop App
La presenza di una **Desktop App indipendente**, con funzionalità simili a quelle del client web, comporta una mancanza di sincronizzazione tra le due applicazioni, portando ad incongruenza dei dati. La presenza di sistemi separati comporta, anche, una doppia implementazione di eventuali aggiornamenti sui diversi sistemi.
- Mancanza di chiarezza e organizzazione nel codice